

KẾ HOẠCH TỔNG THỂ & KIẾN TRÚC HỆ THỐNG

Dự án Mạng lưới Hỗ trợ Khẩn cấp Cộng đồng (SOS Rescue Ecosystem)

Lời mở đầu: Trong các tình huống đe dọa đến tính mạng, mỗi giây trôi qua đều quyết định cơ hội sinh tồn của nạn nhân. Tài liệu này phác thảo kế hoạch chi tiết, kiến trúc kỹ thuật và lộ trình thực hiện ứng dụng khẩn cấp tối ưu hóa hiệu năng bằng ngôn ngữ **Rust** và giao diện phẳng **Flutter**. Đây là một giải pháp toàn diện hướng tới cộng đồng, tối giản thao tác người dùng và tối đa hóa khả năng kết nối ứng cứu tức thời.

TẦM NHÌN CHIẾN LƯỢC: SỨC MẠNH CỦA SỰ KẾT NỐI

Thay vì chỉ gửi tin nhắn thụ động cho người thân (những người có thể ở quá xa), hệ thống sẽ hoạt động như một **Radar sinh tồn**. Ứng dụng kết hợp cơ chế báo động đa tầng: báo cho người thân, thiết lập liên lạc máy chủ đám mây, và kích hoạt mạng lưới ứng cứu xung quanh trong bán kính 500m để có sự trợ giúp vật lý ngay lập tức.

1. Danh sách Tính năng Hệ thống (Chi tiết & Toàn diện)

Để đảm bảo hoạt động hoàn hảo trong mọi điều kiện khắc nghiệt (mất mạng, điện thoại khóa, bị tấn công), ứng dụng được thiết kế với các phân hệ tính năng sau:

A. Phân hệ Kích hoạt SOS Đa phương thức (Không cần mở khóa màn hình)

- Kích hoạt qua Thiết bị Ngoại vi (BLE):** Kết nối ngầm qua Bluetooth Low Energy với nút bấm nhỏ gọn (móc khóa, vòng tay). Nhấn giữ hoặc nhấn đúp nút để kích hoạt SOS từ trong túi quần/áo mà không cần chạm vào điện thoại.
- Lock-screen Widget (Tiếp cận 1 chạm):** Widget khổng lồ hiển thị ngay trên màn hình khóa của iOS và Android. Chỉ cần làm sáng màn hình và bấm trực tiếp, bỏ qua bước nhập mật mã hoặc quét vân tay.
- Kích hoạt bằng Giọng nói (Voice SOS):** Tích hợp Siri Shortcuts và Google Assistant. Người dùng cài đặt từ khóa khẩn cấp cá nhân (Ví dụ: "Cứu tôi với"), hệ thống tự động nhận diện và kích hoạt nhen.

B. Phân hệ Định vị & Xử lý Nền (Tối ưu bằng Rust Core)

- Định vị thông minh tần suất 3 phút/lần:** Khi chế độ SOS kích hoạt, Rust Core kiểm soát GPS, lấy tọa độ chính xác cao gửi lên máy chủ. Thuật toán tự động nhận diện nếu nạn nhân đứng yên (không di chuyển) sẽ giảm tần suất quét để tiết kiệm pin tối đa, và tăng tốc quét khi có sự dịch chuyển.

- **Chế độ Chạy nền Bất tử (Persistent Background Service):** Sử dụng cơ chế Foreground Service kèm thông báo luôn hiển thị trên Android và cơ chế Location Background Updates trên iOS để đảm bảo hệ điều hành không bao giờ tắt ứng dụng khi bộ nhớ đầy.

C. Phân hệ Mạng lưới Radar Cứu hộ Cộng đồng (Tính năng mở rộng quan trọng)

- **Quét tọa độ thời gian thực (Geospatial Radar):** Sử dụng dữ liệu Redis Geo trên server để liên tục cập nhật vị trí ẩn danh của những người dùng khác.
- **Báo động lân cận:** Khi một người bật SOS, toàn bộ các máy cài app trong bán kính 500m - 1km sẽ nhận được thông báo đẩy (Push Notification) mức độ khẩn cấp cao nhất (vượt qua chế độ im lặng/rung của máy) kèm còi hú để họ biết và đến hỗ trợ kịp thời.

D. Phân hệ Thu thập Chứng cứ & Bảo vệ Nạn nhân (Tính năng bổ sung nâng cao)

- **Ghi âm ngầm và Đẩy trực tiếp lên Cloud (Blackbox Recording):** Ngay khi SOS bật, microphone tự động thu âm môi trường xung quanh theo các phân đoạn 30 giây. Các file âm thanh này được mã hóa và đẩy thẳng lên máy chủ theo thời gian thực, đảm bảo giữ lại bằng chứng bảo vệ nạn nhân ngay cả khi điện thoại bị đập nát hoặc tịch thu.
- **Cuộc gọi giả lập (Fake Call Escape):** Tính năng giúp người dùng thoát khỏi tình huống quấy rối nguy hiểm chưa đến mức SOS. Bấm nút kích hoạt nhanh trên widget, máy chủ sẽ thực hiện một cuộc gọi giả đồ chuông như thật để người dùng có lý do thoát thân.

2. Kiến trúc Kỹ thuật & Công nghệ Lựa chọn

THÀNH PHẦN	CÔNG NGHỆ LỰA CHỌN	LÝ DO & VAI TRÒ SỐNG CÒN
Ứng dụng Di động (UI)	Flutter (Dart)	Phát triển đa nền tảng (iOS & Android) nhanh chóng với một mã nguồn duy nhất, giao diện mượt mà, tiếp cận tối đa người dùng.
Lõi Di động (Core Logic)	Rust (via flutter_rust_bridge / UniFFI)	Xử lý định vị ngầm, kết nối Bluetooth ngoại vi ổn định tuyệt đối, quản lý bộ nhớ an toàn (Zero-crash), không gây rò rỉ làm chậm máy.
Hệ thống Máy chủ	Rust (Framework: Axum hoặc Actix-web)	Xử lý hàng triệu kết nối đồng thời với độ trễ cực thấp khi có thảm họa/sự cố diện rộng. Tiết kiệm chi phí vận hành máy chủ đám mây.

THÀNH PHẦN	CÔNG NGHỆ LỰA CHỌN	LÝ DO & VAI TRÒ SỐNG CÒN
Cơ sở Dữ liệu Định vị	Redis (Geospatial) + PostgreSQL	Redis Geo giúp tính toán khoảng cách giữa các người dùng trong bán kính vài trăm mét chỉ trong vài mili-giây, đảm bảo gửi cảnh báo lân cận tức thì.
Cổng Truyền thông SOS	Twilio API / Cổng SMS Quốc gia	Tự động chuyển đổi dữ liệu tọa độ từ Server thành cuộc gọi thoại AI và tin nhắn SMS gửi trực tiếp đến 5 số điện thoại của người thân.

3. Kế hoạch Triển khai từng Giai đoạn & Ước tính Thời gian

Để dự án phát triển một cách bài bản, bền vững và tạo dựng uy tín vững chắc cho lập trình viên, lộ trình được chia làm 4 giai đoạn nghiêm ngặt với tổng thời gian ước tính là **8 tháng**.

Giai đoạn 1: Xây dựng Lõi Sinh tồn (MVP Core) Tháng 1 - Tháng 2

Mục tiêu: Hoàn thiện tính năng cơ bản nhất: Bấm nút trên app -> Gửi vị trí về Server -> Bắn tin nhắn/gọi điện cho người thân.

- Thiết kế giao diện Flutter cơ bản: Màn hình chính có nút SOS lớn, màn hình cấu hình 5 số điện thoại thân nhân.
- Viết Rust Core tích hợp vào ứng dụng di động để xin quyền hệ thống và lấy tọa độ GPS nền.
- Xây dựng Backend Rust (Axum) cơ bản để nhận dữ liệu tọa độ từ thiết bị di động.
- Tích hợp API Twilio trên Server để tự động chuyển tiếp SMS/Call khi nhận tín hiệu khẩn cấp.
- Kết quả:* Sản phẩm chạy thử nghiệm nội bộ ổn định về luồng cứu hộ cốt lõi.

Giai đoạn 2: Kích hoạt Mạng lưới Radar & Tối ưu Pin Tháng 3 - Tháng 5

Mục tiêu: Biến ứng dụng từ cá nhân thành mạng lưới cứu hộ cộng đồng và tối ưu hóa hệ thống chạy ngầm.

- Tích hợp Redis Geospatial vào Backend để định vị vị trí tương đối giữa các tài khoản đang hoạt động.
- Xây dựng hệ thống Push Notification thời gian thực (Firebase Cloud Messaging hoặc giải pháp tự lưu trữ độ trễ thấp).
- Viết thuật toán tối ưu hóa pin bằng Rust: Sử dụng gia tốc kế của điện thoại để nhận biết trạng thái đứng yên/di chuyển, từ đó điều chỉnh thông minh tần suất quét GPS (giảm tiêu thụ pin xuống dưới 2% mỗi ngày ở chế độ chờ).
- Phát triển các Widget hiển thị trên màn hình khóa (Lock-screen Widget) cho cả hai hệ điều hành.

Giai đoạn 3: Tích hợp Ngoại vi & Phân hệ Bảo vệ Nâng cao

Tháng 6 - Tháng 7

Mục tiêu: Đa dạng hóa cách thức kích hoạt bí mật và thu thập bằng chứng bảo vệ nạn nhân.

- Lập trình thư viện Bluetooth LE trong Rust Core để quét và duy trì kết nối ngầm ổn định với các loại nút bấm thông minh phổ biến trên thị trường.
- Phát triển tính năng "Blackbox Recording": Kích hoạt Microphone ngầm, ghi âm, mã hóa dữ liệu ngay trên thiết bị bằng thuật toán mã hóa đầu cuối (End-to-End Encryption) trước khi tải lên Cloud.
- Xây dựng tính năng cuộc gọi giả lập (Fake Call) điều khiển từ xa qua Server đám mây.
- Tích hợp cơ chế nhận diện giọng nói nền qua API trợ lý ảo mặc định của thiết bị.

Giai đoạn 4: Kiểm thử Tải, Bảo mật & Phát hành Cộng đồng

Tháng 8

Mục tiêu: Đảm bảo hệ thống không sập khi có hàng vạn người dùng đồng thời và đưa ứng dụng lên các kho ứng dụng chính thức.

- Tiến hành Stress Test (Kiểm thử tải giới hạn) cho Backend Rust nhằm giả lập tình huống 100.000 yêu cầu SOS gửi lên cùng một giây.
- Kiểm thử xâm nhập (Penetration Testing) để đảm bảo dữ liệu vị trí ẩn danh của người dùng không bị rò rỉ hoặc bị kẻ xấu lợi dụng để theo dõi ngược.
- Đưa ứng dụng lên Google Play Store và Apple App Store (Giải trình và xin các quyền nhạy cảm như Định vị nền, Gửi thông báo khẩn cấp vượt rào âm thanh).
- Công bố mã nguồn mở (Open-source) phần Core trên GitHub để tạo sự minh bạch, thu hút cộng đồng lập trình viên cùng đóng góp và xây dựng uy tín lớn cho dự án.

4. Những thách thức lớn và Biện pháp Quản trị Rủi ro

Vì đây là ứng dụng liên quan trực tiếp đến tính mạng con người, mọi rủi ro kỹ thuật đều phải có phương án dự phòng:

1. **Báo động giả (False Alarms):** Để tránh việc người dùng vô tình chạm phải nút SOS gây hoang mang cho cộng đồng, hệ thống sẽ thiết lập cơ chế 5 giây đếm ngược kèm rung phản hồi (Haptic Feedback) mạnh mẽ trước khi phát tín hiệu chính thức. Để hủy trạng thái SOS sau khi đã phát, người dùng bắt buộc phải nhập mã PIN bảo mật cá nhân.
2. **Sự nghiêm ngặt của Apple/Google:** Việc xin quyền định vị nền liên tục (`Always Allow`) và quyền can thiệp âm thanh hệ thống cực kỳ khó khăn trên iOS và Android. Giải pháp là chuẩn bị một bộ hồ sơ kỹ thuật chi tiết chứng minh ứng dụng phục vụ mục đích cứu hộ phi lợi nhuận cho cộng đồng để được phê duyệt đặc cách.
3. **Môi trường không có Internet (Mất sóng 3G/4G):** Khi thiết bị không thể kết nối tới Server, Rust Core sẽ tự động chuyển hướng, cố gắng kích hoạt cổng SMS nội tại của thiết bị (nếu là Android) để nhắn tin trực tiếp đến 5 số điện thoại thân nhân như một phương án cuối cùng.

**Tài liệu này đóng vai trò là kim chỉ nam cho toàn bộ vòng đời phát triển của dự án.
Sự kết hợp giữa hiệu năng tối cao của Rust và tốc độ triển khai của Flutter sẽ tạo nên một lá
chấn sinh tồn công nghệ vững chắc cho cộng đồng.**